

Ali Parchekani

Teaching Philosophy Statement

My teaching philosophy is grounded in a simple idea: students learn technical subjects best when they are asked to do the work of the discipline in carefully supported steps. In engineering, statistics, computer science, game theory, and artificial intelligence, students do not only need definitions. They need to recognize structure, question assumptions, test ideas against evidence, and explain their reasoning clearly. My role as an instructor is to design learning experiences in which abstract ideas become usable: start from a concrete problem, name the underlying concept, then apply it to a new situation.

My experience has shaped this philosophy across a wide range of classrooms. As a Ph.D. candidate in Electrical and Computer Engineering at the University of Toronto, I have taught and supported courses in statistics, probability, Bayesian statistics, stochastic processes, machine learning, artificial intelligence, programming, computer networks, calculus, linear algebra, physics, signals and systems, and game theory. I have also taught pre-university students in courses such as Critical Thinking and Statistics and Games, Strategy, and AI. This breadth has taught me that student difficulty is often not a lack of ability; it is a gap between the expert's compressed explanation and the student's current mental model. Good teaching identifies that gap and builds a path across it.

I therefore teach from experience toward abstraction. In a statistics class, this means beginning with a dataset, a claim, or a misleading graph before introducing formal vocabulary. In a networks or engineering course, it means starting from a real system, diagram, packet trace, signal, or design constraint before moving to equations. In Games, Strategy, and AI, I designed lessons around play, discussion, and creation: students first played simple games, then analyzed rules, choices, incentives, feedback, fairness, and algorithms, and finally built their own games. Students see the phenomenon first, learn the language needed to describe it, and then use that language to reason more precisely.

Active learning is central to my classroom, but I do not treat activity as entertainment. Activity must create evidence for thinking. I use short explanations, guided worksheets, small-group problem solving, discussion, office-hour style support, and project checkpoints to keep students working with the material rather than simply watching it. In Games, Strategy, and AI, students moved from paper prototypes to playable GDevelop browser games with one visible AI capability, such as a rule-based non-player character, a pattern-tracking opponent, or adaptive difficulty. The project required them to represent systems with rules, variables, flowcharts, playtest evidence, and revision notes. The goal was not only to produce a game, but to understand how a decision system works and how it can be improved.

I also believe that rigorous teaching must be inclusive. Many of my students, especially in pre-university settings, are working in English as an additional language while also learning unfamiliar academic content. I separate clarity of thinking from perfection of English. I give students vocabulary, sentence frames, diagrams, examples, and structured opportunities to rehearse ideas with a partner before speaking to the whole class. I ask students to explain why, but I also help them find the words for that explanation. My aim is to create a classroom where students are challenged intellectually while still feeling allowed to try, revise, and ask basic questions without embarrassment.

In technical courses, confidence and precision have to grow together. A student who can solve a problem but cannot explain the assumptions behind the solution is not yet fully fluent in the subject. A student who can speak confidently but does not test claims against evidence also needs support. I try to build both habits at once. In tutorials and office hours, I often focus on the reasoning move behind a solution: What information is available? What is the objective? What assumption are we making? What would change if the condition

changed? These questions help students transfer knowledge beyond the exact problem they have already seen.

My approach to assessment follows the same principle. Assessment should reward explanation, application, and revision, not only final answers. I use projects, reports, presentations, guided problem sets, and rubrics that make the criteria visible. When students build or analyze a system, I want them to show the process: the first attempt, the evidence from testing, the revision, and the reason the revision matters. In a project-based class, this might mean requiring playtest evidence and a written explanation of design changes. In a statistics or machine learning course, it might mean asking students to justify a model choice, interpret an error, or explain what a result does and does not prove. Constructive feedback is essential because students need to know not only whether they are correct, but how to improve their reasoning.

I see teaching as an iterative design process. During course delivery, I pay attention to where students slow down, where instructions are unclear, and where the planned lesson no longer fits the room. In my Games, Strategy, and AI planning, for instance, I restructured later sessions when students needed more time to make their digital projects work, converting lecture time into structured studio sprints while preserving the conceptual goals of the course. A good instructor protects the learning outcome, not the exact script.

Ultimately, I want students to leave my courses with more than content knowledge. I want them to leave with a way of approaching unfamiliar technical problems: identify the system, define the variables, examine the assumptions, test the idea, communicate the evidence, and revise when reality pushes back. Whether students go on to study engineering, computer science, statistics, data science, or another field, this habit of disciplined reasoning will serve them. My goal is to help students become clearer thinkers, more confident problem solvers, and more responsible users of technical knowledge.